



PROGRAMAÇÃO LINEAR COM BIBLIOTECAS DO PYTHON

JOÃO CARLOS FERREIRA MARTINS.

LUIZ HENRIQUE SANTOS DIAS.

OTIMIZAÇÃO I



SUMÁRIO

01 Programação Linear

02 Biblioteca

03 CBC Solver

04 Método Simplex

05 Simplex de 2 Fases

06 Simplex Dual

07 Programação Linear Inteira

08 Conclusão

PROGRAMAÇÃO LINEAR

- Técnica da Pesquisa Operacional utilizada em problemas de otimização
- Busca maximizar ou minimizar uma função objetivo.
- Todas as funções envolvidas são lineares

Maximizar

$$L = 4x_1 + x_2$$

Sujeito à:

$$9x_1 + x_2 \leq 18$$

$$3x_1 + x_2 \leq 12$$

$$x_1, x_2 \geq 0$$



BIBLIOTECAS EM PYTHON

Python possui diversas bibliotecas que permitem modelar e resolver problemas de Programação Linear e Inteira de forma eficiente.

Entre as mais utilizadas estão PuLP, SciPy (linprog), Pyomo e OR-Tools, que oferecem recursos para definição de variáveis, funções objetivo e restrições.

Essas ferramentas podem utilizar algoritmos como Simplex, Dual Simplex e Interior Point, sendo usadas como formatadores de PPL e disponibilizando-os para diferentes solvers.

PULP

O PuLP utiliza uma abordagem baseada em funções e objetos para modelar problemas de otimização.

As variáveis, função objetivo e restrições são definidas de forma semelhante à formulação matemática tradicional.

Código mais intuitivo e fácil de interpretar.

```
from pulp import *
```

LINPROG (SCIPY)

O linprog utiliza uma abordagem baseada em matrizes e vetores.

A função objetivo e as restrições são representadas por coeficientes numéricos em matrizes.

Essa forma de modelagem mais próxima da utilizada em algoritmos de otimização.

Resolve problemas de minimização

```
from scipy.optimize import linprog
```


FUNÇÕES DO PULP

LpProblem()

Cria o modelo de otimização, definindo se o problema será de maximização (LpMaximize) ou minimização (LpMinimize).

```
prob = LpProblem("Exemplo", LpMaximize)
```

solve()

Executa o solver escolhido para encontrar a solução ótima do problema.

```
prob.solve()
```

LpVariable()

Cria as variáveis de decisão do problema, permitindo definir limites inferiores, superiores

```
x = LpVariable('x', lowBound=0)  
y = LpVariable('y', lowBound=0)
```

writeLP()

Exporta o modelo de Programação Linear para um arquivo .lp

```
\* Exemplo *\nMaximize\nOBJ: 3 x + 2 y\nSubject To\n_C1: x + y <= 10\n_C2: x <= 6\nEnd
```

RESULTADOS NO PULP

```
print("x1 =", value(x1))  
print("x2 =", value(x2))
```

Visualiza o valor ótimo das
variáveis definidas

```
print("Z =", value(model.objective))
```

Visualiza o valor da função
objetivo

```
print("Status:", LpStatus[model.status])
```

Visualiza o status final do PPL

Optimal / Not Solved / Infeasible / Unbounded / Undefined

PARÂMETROS DO LINPROG

```
res = linprog(c, A_ub=A, b_ub=b, bounds=bounds)
```

```
c = [-3, -2] # Z = -3x -2y
```

Vetor com os coeficientes da função objetivo.

```
A = [  
    [1, 1], # x + y <= 10  
    [1, 0]  # x <= 6  
]
```

Matriz das restrições de desigualdade do tipo $Ax \leq b$.

```
b = [10, 6]
```

Vetor dos limites das restrições de desigualdade.

```
bounds = [(0, None), (0, None)]
```

Define os limites inferior e superior das variáveis.

RESULTADOS NO LINPROG

```
print("x =", res.x[0])  
print("y =", res.x[1])
```

Visualiza o valor ótimo das
variáveis definidas

```
print("valor ótimo =", -res.fun)
```

Visualiza o valor da função
objetivo

```
print(res.success)
```

Visualiza o status final do PPL

True / False

EXEMPLO NO PYTHON

Maximizar $Z = 3x_1 + 2x_2$

Sujeito à:

$$x_1 + x_2 \leq 10$$

$$x_1 \leq 6$$

$$x_1, x_2 \geq 0$$

SOLVER CBC

É um solver open source de otimização matemática.

Como o PuLP utiliza o CBC ?

1. Código Python
2. PuLP cria o modelo matemático
3. PuLP converte para formato LP/MPS
4. CBC recebe o problema
5. CBC resolve o problema
6. Resultado retorna ao Python

SIMPLEX

O Método Simplex é um algoritmo desenvolvido para resolver problemas de programação linear, encontrando a melhor solução para uma função objetivo sujeita a restrições lineares.

O método percorre sucessivamente os vértices da região viável, avaliando soluções básicas até alcançar a solução ótima.

É amplamente utilizado em problemas de maximização e minimização envolvendo alocação de recursos, planejamento da produção e logística.

SIMPLEX

Exemplo Resolvido

Maximizar $Z = 3x_1 + 5x_2$

Sujeito à:

$$x_1 + 2x_2 \leq 8$$

$$3x_1 + 2x_2 \leq 12$$

$$x_1, x_2 \geq 0$$

```
# VARIÁVEIS DE DECISÃO
x1 = LpVariable("x1", lowBound=0)
x2 = LpVariable("x2", lowBound=0)

# FUNÇÃO OBJETIVO (MAXIMIZAR)
model = LpProblem("Simplex_Primal", LpMaximize)
model += 3*x1 + 5*x2

# RESTRIÇÕES
model += x1 + 2*x2 <= 8, "Restricao_1"
model += 3*x1 + 2*x2 <= 12, "Restricao_2"

# SOLVER (SIMPLEX)
solver = PULP_CBC_CMD(msg=False)

# RESOLVER O PROBLEMA
model.solve(solver)

# RESULTADOS
print("Status:", LpStatus[model.status])
print("x1 =", value(x1))
print("x2 =", value(x2))
print("Z =", value(model.objective))
```


SIMPLEX DE DUAS FASES

O Simplex de Duas Fases é uma variação do algoritmo Simplex utilizada para resolver problemas de programação linear que não possuem uma solução básica viável inicial evidente.

Na primeira fase, são introduzidas variáveis artificiais e minimizada uma função auxiliar para encontrar uma solução viável para o problema original.

Na segunda fase, as variáveis artificiais são removidas e o Método Simplex é aplicado para determinar a solução ótima.

SIMPLEX DE DUAS FASES

Exemplo Resolvido

Maximizar $Z = 2x_1 + 3x_2$

Sujeito à:

$$x_1 + x_2 = 4$$

$$x_1 + 2x_2 \geq 6$$

$$x_1, x_2 \geq 0$$

```
# VARIÁVEIS DE DECISÃO
x1 = LpVariable("x1", lowBound=0)
x2 = LpVariable("x2", lowBound=0)

# FUNÇÃO OBJETIVO (MINIMIZAR)
model = LpProblem("Duas_Fases", LpMinimize)
model += 2*x1 + 3*x2

# RESTRIÇÕES
model += x1 + x2 == 4, "Restricao_1"
model += x1 + 2*x2 >= 6, "Restricao_2"

# SOLVER
solver = PULP_CBC_CMD(msg=False)

# RESOLVER O PROBLEMA
model.solve(solver)

# RESULTADOS
print("Status:", LpStatus[model.status])
print("x1 =", value(x1))
print("x2 =", value(x2))
print("Z =", value(model.objective))
```

SIMPLEX DUAL

O Método Simplex Dual é uma variação do algoritmo Simplex utilizada para resolver problemas de programação linear.

Diferente do Simplex Primal, o Dual Simplex inicia com uma solução primal inviável, mas dual viável, realizando iterações até encontrar a solução ótima.

PRIMAL	DUAL
MINIMIZAÇÃO	MAXIMIZAÇÃO
CUSTOS	PREÇOS SOMBRA

SIMPLEX DUAL

Primal

Minimizar $Z = 4x_1 + 5x_2$

Sujeito à:

$$x_1 + 4x_2 \geq 5$$

$$3x_1 + 2x_2 \geq 4$$

$$x_1, x_2 \geq 0$$

```
print("PRIMAL")

model = LpProblem("Primal", LpMinimize)

x1 = LpVariable("x1", lowBound=0)
x2 = LpVariable("x2", lowBound=0)

model += 4*x1 + 5*x2

model += x1 + 4*x2 >= 5, "Restricao_1"
model += 3*x1 + 2*x2 >= 4, "Restricao_2"

model.solve(PULP_CBC_CMD(msg=False))
```

SIMPLEX DUAL

Dual

Maximizar $Z = 5y_1 + 4y_2$

Sujeito à:

$$y_1 + 3y_2 \leq 4$$

$$4y_1 + 2y_2 \leq 5$$

$$y_1, y_2 \geq 0$$

```
# PROBLEMA DUAL
print("DUAL")

# CRIAÇÃO DO MODELO
model = LpProblem("Dual", LpMaximize)

# VARIÁVEIS
y1 = LpVariable("y1", LowBound=0)
y2 = LpVariable("y2", LowBound=0)

# FUNÇÃO OBJETIVO
model += 5*y1 + 4*y2

# RESTRIÇÕES
model += y1 + 3*y2 <= 4
model += 4*y1 + 2*y2 <= 5

# RESOLVER
model.solve(PULP_CBC_CMD(msg=True))
```

SIMPLEX DUAL

Análise Dual

ANÁLISE DUAL

Restricao_1

Preço-sombra = 0.70

Folga = -0.00

Restricao_2

Preço-sombra = 1.10

Folga = -0.00

- Preços-sombra indicam o impacto das restrições no valor ótimo.
- Preço-sombra da Restrição 1: 0,70
- Preço-sombra da Restrição 2: 1,10
- Folgas iguais a 0 mostram que as restrições estão ativas.

PROGRAMAÇÃO LINEAR INTEIRA

A Programação Linear Inteira é uma técnica de otimização matemática em que as variáveis de decisão devem assumir valores inteiros.

Características

- Extensão da Programação Linear.
- Possui função objetivo e restrições.
- Os modelos podem possuir variáveis inteiras, binárias ou mistas.

Exemplo

Não é possível produzir:

- 2.7 Caminhões
- 4.3 Aviões
- 1.5 Funcionários

PROGRAMAÇÃO LINEAR INTEIRA

Aplicações

- Transporte e logística;
- Planejamento de produção;
- Escalonamento;
- Problema da mochila;
- Caixeiro viajante.

Resolução

Problemas de PLI normalmente utilizam algoritmos como:

- Branch and Bound;
- Branch and Cut.

ENUNCIADO DO PROBLEMA

Uma companhia aérea deseja distribuir aeronaves entre aeroportos e destinos minimizando custos operacionais.

Cada aeroporto possui uma quantidade limitada de aeronaves disponíveis, e cada destino possui uma capacidade máxima de recebimento de aeronaves.

A empresa possui três aeroportos principais de origem:

- Aeroporto A
- Aeroporto B
- Aeroporto C

Cada aeroporto possui uma quantidade limitada de aeronaves disponíveis:

Aeroporto	Aeronaves Disponíveis
A	300
B	400
C	200

Essas aeronaves precisam atender às demandas de quatro destinos:

Destino	Demanda de Aeronaves
Destino 1	200
Destino 2	150
Destino 3	250
Destino 4	200

O custo operacional para deslocar uma aeronave entre cada aeroporto de origem e cada destino é apresentado na tabela abaixo:

Origem	Destino 1	Destino 2	Destino 3	Destino 4
Aeroporto A	8	6	10	7
Aeroporto B	9	12	8	11
Aeroporto C	14	7	10	12

MODELAGEM DO PROBLEMA DA COMPANHIA AÉREA

OBJETIVO

Minimizar o custo total de transporte de aeronaves entre aeroportos e destinos.

VARIÁVEIS DE DECISÃO

$$X_{ij}$$

Onde:

X_{ij} Quantidade de aeronaves enviadas do aeroporto i para o destino j

FUNÇÃO OBJETIVO

MINIMIZAR:

$$Z = 8x_{A1} + 6x_{A2} + 10x_{A3} + 7x_{A4} + 9x_{B1} + 12x_{B2} + 8x_{B3} + 11x_{B4} + 14x_{C1} + 7x_{C2} + 10x_{C3} + 12x_{C4}$$

RESTRIÇÕES DE OFERTA:

Aeroporto A: $x_{A1} + x_{A2} + x_{A3} + x_{A4} \leq 300$

Aeroporto B: $x_{B1} + x_{B2} + x_{B3} + x_{B4} \leq 400$

Aeroporto C: $x_{C1} + x_{C2} + x_{C3} + x_{C4} \leq 200$

FUNÇÃO OBJETIVO

MINIMIZAR:

$$Z = 8x_{A1} + 6x_{A2} + 10x_{A3} + 7x_{A4} + 9x_{B1} + 12x_{B2} + 8x_{B3} + 11x_{B4} + 14x_{C1} + 7x_{C2} + 10x_{C3} + 12x_{C4}$$

RESTRIÇÕES DA DEMANDA:

$$\text{Destino 1: } x_{A1} + x_{B1} + x_{C1} \geq 200$$

$$\text{Destino 2: } x_{A2} + x_{B2} + x_{C2} \geq 150$$

$$\text{Destino 3: } x_{A3} + x_{B3} + x_{C3} \geq 250$$

$$\text{Destino 4: } x_{A4} + x_{B4} + x_{C4} \geq 200$$

PROGRAMAÇÃO LINEAR INTEIRA

```
# MODELO
model = LpProblem("PLI_Branch_And_Bound",LpMinimize)

# ROTAS
routes = [
    (a, d)
    for a in airports
    for d in destinations
]

# VARIÁVEIS INTEIRAS
shipments = LpVariable.dicts("Enviar",routes,lowBound=0,cat='Integer')

# FUNÇÃO OBJETIVO
model += lpSum(
    costs[a, d] * shipments[a, d]
    for (a, d) in routes
)
```


CONCLUSÃO

Python facilita a modelagem e resolução de problemas de Programação Linear.

Bibliotecas como PuLP e linprog reduzem a complexidade de implementação dos algoritmos.

A automação dos cálculos aumenta a eficiência e a confiabilidade das soluções obtidas.

Ferramentas de otimização possuem ampla aplicação em problemas acadêmicos e industriais.

REPOSITÓRIO

GITHUB

CONTATOS

luizhenriquemc15@gmail.com

joaocarlos12349@gmail.com

REFERÊNCIAS

MITCHELL, Stuart; O'SULLIVAN, Michael; DUNNING, Iain.

PuLP: a linear programming toolkit for Python. Optimization Online, 2011. Disponível em: A Programação Linear Inteira é uma técnica de otimização matemática em que as variáveis de decisão devem assumir valores inteiros..

PuLP Documentation. Disponível em: <https://coin-or.github.io/pulp/>.

SCIPY DEVELOPERS.

scipy.optimize.linprog — SciPy Documentation. Disponível em: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>. Acesso em: 24 maio 2026.

DATA CAMP. Linear Programming in Python. Disponível em: <https://www.datacamp.com/pt/tutorial/linear-programming>. Acesso em: 15 maio 2026

COIN-OR CBC Solver. Disponível em: <https://github.com/coin-or/Cbc>

COIN-OR. Computational Infrastructure for Operations Research. Disponível em: <https://www.coin-or.org/>

OPENAI. ChatGPT. Modelo de linguagem baseado em inteligência artificial. Disponível em: <https://chat.openai.com/>



OBRIGADO PELA ATENÇÃO!!

DÚVIDAS ?!